Clasificación automática de 104 tipos de flores con redes neuronales profundas⋆

Angel Eduardo-Luna[a,1], Andrea Estefanía De la Peña Contreras[a,2], Jannette Cristina Flores Rodríguez[a,3], Rey Iván de la Fuente Chávez[a,4] and Andrés Huerta Robinson[a,5]

[a]*Escuela de Ingeniería y Ciencias, Tecnológico de Monterrey, Monterrey, Nuevo León, México*

INFORMACIÓN

Palabras clave:
Clasificación de imágenes
Redes neuronales convolucionales
Curvas de aprendizaje
Destilación de datos
F1 macro

Resumen

Este trabajo aborda la clasificación automática de 104 tipos de flores mediante redes neuronales convolucionales (CNN) y técnicas de destilación de datos. Se utiliza el conjunto de imágenes de la competencia *Petals to the Metal* de Kaggle, compuesto por 23,847 fotografías RGB distribuidas entre entrenamiento, validación y prueba. La similitud visual entre algunas clases, las variaciones de iluminación y el desbalance de los datos representan desafíos importantes para la clasificación.

Se implementó una CNN de cuatro bloques convolucionales (480,936 parámetros), entrenada mediante retropropagación con el optimizador Adam, parada temprana por F1 macro y guardado del mejor modelo. Además, se aplicó *Distribution Matching* (DM) para generar conjuntos reducidos de imágenes sintéticas, y se comparó el entrenamiento con el conjunto completo, con subconjuntos aleatorios de imágenes reales y con conjuntos destilados de una y diez imágenes por clase.

Con el conjunto completo, el modelo alcanzó un F1 macro de 0.5538 y una exactitud de 61.61 % en validación. Las curvas de aprendizaje muestran que, a partir de la época 25, la pérdida de validación se estanca mientras la de entrenamiento sigue bajando, y que los conjuntos reducidos memorizan sus pocas imágenes. Con el conjunto completo se evaluaron tres variaciones de hiperparámetros (tasa de aprendizaje, *dropout* y regularización): reducir la tasa de aprendizaje a 3×10^{-4} produjo subajuste (F1 de 0.4605) y quitar la regularización aumentó el sobreajuste (F1 de 0.5332), mientras que la configuración de referencia obtuvo el mejor F1, 0.5784. Todas las tablas y figuras se reproducen con el notebook público del proyecto.

1. Contexto del problema

La diversidad del mundo natural es enorme: se estima que existen más de 5,000 especies de mamíferos, 10,000 de aves y 30,000 de peces, pero hay más de 400,000 tipos de flores (Kaggle, 2020). Distinguir entre ellas puede requerir conocimiento especializado y, aun para una persona con experiencia, muchas flores son visualmente muy parecidas entre sí. Identificarlas de forma manual es lento y no escala cuando se tienen miles de fotografías. Por eso resulta valioso contar con modelos que reconozcan el tipo de flor de forma automática a partir de una imagen.

Este proyecto parte de la competencia *Petals to the Metal*, de Kaggle, cuyo reto es construir un modelo que identifique el tipo de flor en un conjunto de imágenes. La competencia se limita a 104 clases de flores y consiste en un problema de clasificación multiclase de imágenes en el que el modelo debe captar detalles finos de forma, estructura y color para distinguir entre categorías cercanas.

Este problema tiene varias dificultades que lo hacen relevante desde un enfoque de redes neuronales. Primero, las clases son heterogéneas, ya que algunas solo tienen un subtipo de flor y otras agrupan muchas subvariedades. Además, las imágenes no fueron capturadas en el mismo entorno, lo que les da más variedad. Por último, el número de imágenes por clase tampoco es uniforme, por lo que, considerando la métrica de evaluación, es importante que el modelo funcione bien tanto en las clases con pocos ejemplos como en las que tienen muchos.

2. Descripción del dataset

2.1. Origen

Los datos vienen de *Petals to the Metal – Flower Classification on TPU*, una competencia *Getting Started* de Kaggle cuyo objetivo es aprender a entrenar modelos en TPU (Kaggle, 2020). Según la descripción de la competencia, las imágenes se tomaron de cinco conjuntos de datos públicos. Algunas clases son muy específicas (por ejemplo, *pink primrose*, que contiene un solo subtipo de flor) y otras agrupan muchas subvariedades (por ejemplo, *wild rose*).

Al revisar la lista de clases nos dimos cuenta de que las primeras 102 siguen exactamente el orden del conjunto Oxford 102 Flowers (Nilsback & Zisserman, 2008): 94 conservan el mismo nombre y las otras 8 se renombraron, se corrigieron o se ampliaron (por ejemplo, *oxeye daisy* pasó a ser *daisy* y *bearded iris* pasó a ser *iris*). Las dos clases extra son *common tulip* y *wild rose*. Las etiquetas están en inglés, así que usamos los nombres originales en cursiva.

2.2. Tamaño y estructura

En total hay 23,847 imágenes a color (RGB) de 104 clases, repartidas como muestra la Tabla 1. Vienen en formato

⋆ Situación problema: Avance 3 (versión corregida). Diseño de Redes Neuronales y Aprendizaje Profundo (TC2035.201). Proyecto basado en la competencia *Petals to the Metal – Flower Classification on TPU* de Kaggle. Código y notebook: https://github.com/Lunaaaalj/NeuralNetworksTeamEnvironment.

[1]Matrícula: A00842657
[2]Matrícula: A00840973
[3]Matrícula: A01255711
[4]Matrícula: A01709445
[5]Matrícula: A00838626

Tabla 1
Particiones del conjunto de datos.

Partición	Imágenes	%	¿Etiquetado?
Entrenamiento	12,753	53.5	Sí
Validación	3,712	15.6	Sí
Prueba	7,382	31.0	No
Total	23,847	100.0	

TFRecord, que junta muchas imágenes en pocos archivos binarios para que la TPU las lea rápido. Cada registro tiene tres campos: `image` (la foto codificada en JPEG), `id` (un identificador único) y `class` (la etiqueta, un entero de 0 a 103). Los registros de prueba no traen `class`, porque esas etiquetas solo las tiene Kaggle para calificar.

Cada partición viene en 4 resoluciones: 192×192, 224×224, 331×331 y 512×512 píxeles, con 16 archivos TFRecord por partición, por lo que la base de datos trae 192 archivos TFRecord (4 resoluciones × 3 particiones × 16 archivos) más `sample_submission.csv`. Tener varias resoluciones conviene porque no todas las arquitecturas se diseñaron para el mismo tamaño de entrada: ResNet50, DenseNet201 y EfficientNetB0 usan 224×224, mientras que las EfficientNet más grandes usan resoluciones mayores. Para dimensionar el problema, una sola imagen de 224×224 ya tiene 150,528 valores.

2.3. Ejemplos de las imágenes

La Tabla 2 muestra algunas clases concretas y lo que cada una nos dice sobre el problema. Además, dentro de una misma clase, las fotos cambian mucho de escala, pose e iluminación (Nilsback & Zisserman, 2008). La Figura 1 muestra fotos reales del conjunto de entrenamiento.

2.4. Balance de clases y métrica de evaluación

Con 12,753 imágenes de entrenamiento y 104 clases, tenemos en promedio 123 imágenes por clase ($12{,}753/104 \approx 122.6$). Sin embargo, las clases no tienen el mismo tamaño: como unas clases agrupan muchas subvariedades y otras solo una, algunas tienen muchas más imágenes que otras (de 18 a 782 imágenes de entrenamiento por clase).

Por este desbalance, la exactitud no basta: un modelo puede acertar mucho en las clases grandes, fallar en las clases pequeñas y aún verse bien. Por eso Kaggle califica con el F1 macro (Kaggle, 2020). Para cada clase c se calcula la precisión P_c, la sensibilidad (*recall*) R_c y su media armónica:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \tag{1}$$

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c}, \qquad F1_{\text{macro}} = \frac{1}{104}\sum_{c=0}^{103} F1_c, \tag{2}$$

donde TP_c, FP_c y FN_c son los verdaderos positivos, falsos positivos y falsos negativos de la clase c. Así, todas las clases pesan lo mismo, sin importar cuántas imágenes tengan.

2.5. Implicaciones para el proyecto

Como Kaggle ya separa entrenamiento y validación, no necesitamos hacer nuestra propia partición: entrenamos con `train`, elegimos modelos e hiperparámetros con `val` y reservamos `test` para la predicción final. Como las clases están desbalanceadas, siempre reportamos el F1 macro y no solo la exactitud.

3. Algoritmos y aplicaciones

Dado el problema planteado, su solución implica el uso de algoritmos de visión por computadora.

3.1. Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales (CNN) son una alternativa especialmente adecuada para este problema debido a su capacidad para procesar información espacial. Mediante filtros convolucionales pueden aprender automáticamente características de las imágenes, como bordes, texturas y formas (Figura 2). En las capas internas pueden emplearse funciones de activación como ReLU, mientras que Softmax puede utilizarse en la salida para representar las probabilidades de las distintas clases.

Este método parece ser el más adecuado para obtener buenos resultados; sin embargo, por la cantidad de datos y de operaciones involucradas, es lento y computacionalmente pesado. Esto abre la posibilidad de usar otros algoritmos que se ajusten a las necesidades del problema.

3.2. Otros algoritmos sin aprendizaje profundo

k vecinos más cercanos (KNN) Representa cada imagen como un vector de características, mide la similitud entre puntos (normalmente con la distancia euclidiana) y clasifica la imagen según sus k vecinos más cercanos. Es un algoritmo de aprendizaje supervisado.

Bosque aleatorio Combina un conjunto de árboles de decisión y clasifica según el resultado de la mayoría de ellos.

Máquinas de vectores de soporte (SVM) Encuentran el hiperplano óptimo para separar dos clases. La variante multiclase combina varios clasificadores binarios, como se requiere en este contexto.

3.3. Otros modelos de aprendizaje profundo

ResNet Conocida también como *residual network*. Útil para entrenar redes neuronales muy profundas. Su principal limitante es la capacidad de nuestro *hardware* o el acceso a equipo capaz de entrenar este tipo de redes.

Otros modelos ya implementados Como **AlexNet** o **GoogLeNet/Inception**, que tienen una gran capacidad para reconocer y clasificar imágenes (IBM, s.f.).

3.4. Aplicaciones posibles

Un modelo capaz de clasificar tipos de flores a partir de imágenes puede tener distintas aplicaciones reales. Una de ellas es la **identificación automática de plantas**, donde

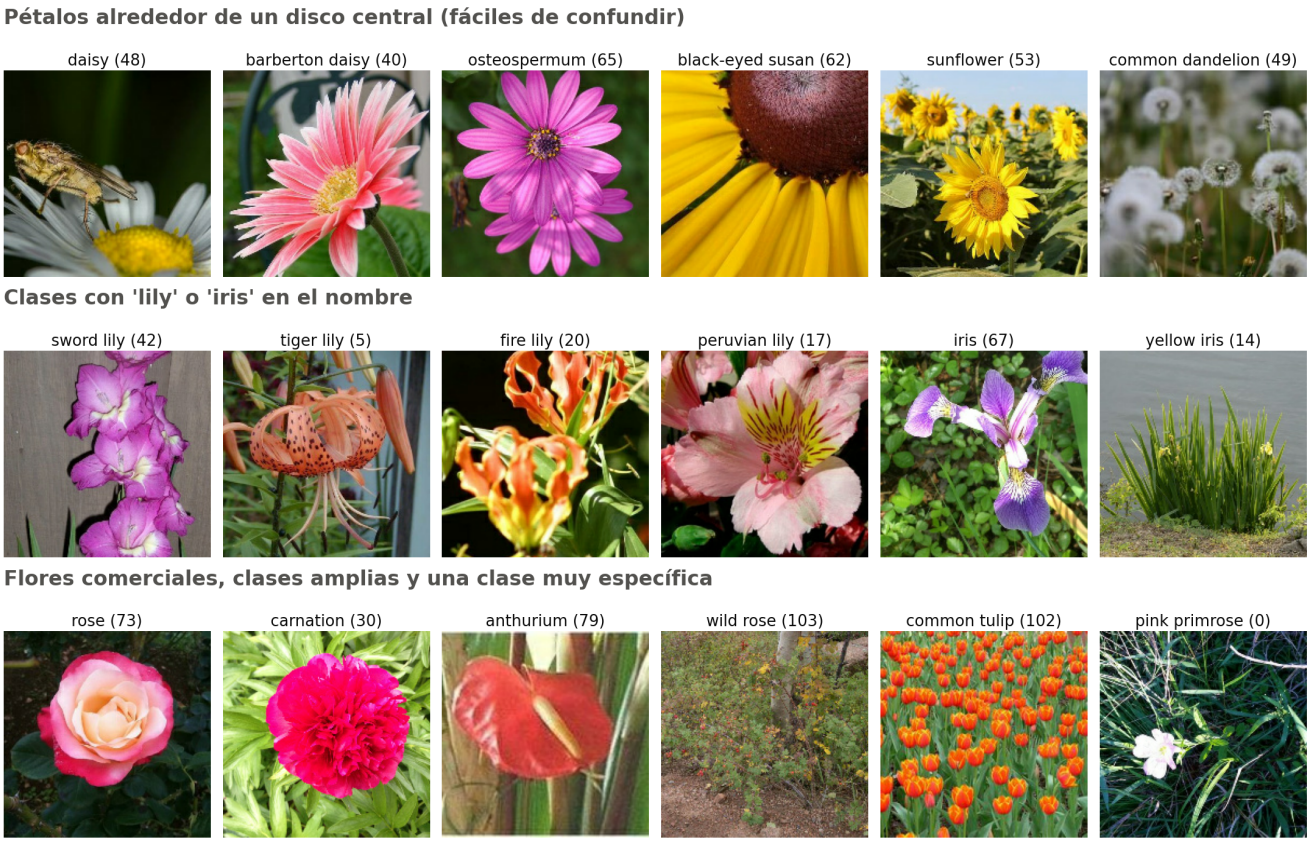


Figura 1: Ejemplos reales del conjunto de entrenamiento con su clase y número de etiqueta.

Tabla 2
Clases de ejemplo (nombre y etiqueta) y lo que ilustran.

Clase (etiqueta)	Qué ilustra
sunflower (53)	Girasol. Flor grande y muy característica. Posiblemente sea de las más fáciles de identificar.
daisy (48), *barberton daisy* (40), *osteospermum* (65), *black-eyed susan* (62)	Margarita, gerbera, osteospermum y rudbeckia. Todas tienen pétalos alrededor de un disco central. Pueden surgir confusiones entre ellas.
sword lily (42), *tiger lily* (5), *fire lily* (20), *peruvian lily* (17)	Gladiola, lirio tigre, gloriosa y alstroemeria. Todas dicen *lily*, pero son de cuatro familias botánicas distintas: el modelo tiene que fijarse en la forma, no en el nombre.
iris (67), *yellow iris* (14)	Misma forma general, la diferencia principal es el color.
rose (73), *wild rose* (103)	Dos clases de rosas. *Wild rose* es amplia (varias subvariedades) y no viene de Oxford 102.
common tulip (102)	Tulipán. Clase amplia que tampoco viene de Oxford 102.
pink primrose (0)	Clase muy específica con un solo subtipo de flor.
carnation (30)	Clavel, una de las flores comerciales que se producen en el Estado de México.

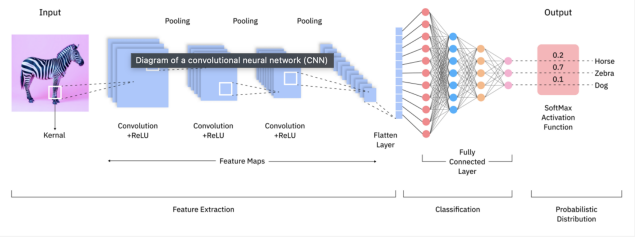


Figura 2: La convolución descompone la imagen en características para aprender a clasificar (IBM, s.f.).

un usuario toma una fotografía y recibe como resultado la especie o clase más probable.

Un ejemplo de este tipo de aplicaciones está en el apoyo a plataformas de ciencia ciudadana y el monitoreo de biodiversidad. Por ejemplo, iNaturalist ha acumulado más de 300 millones de observaciones hechas por personas comunes (iNaturalist, 2026) y utiliza un modelo de visión por computadora, entrenado con esas mismas observaciones, para sugerir la especie a partir de una fotografía. Sin este apoyo automático, cada observación dependería de que una persona la revisara detenidamente, lo que haría el proceso lento y difícil de escalar. Este proyecto es una versión a menor escala de este tipo de sistemas.

Otra aplicación sería como **herramienta educativa**, por ejemplo, en aplicaciones para estudiantes o visitantes de jardines botánicos que permitan reconocer flores y consultar información sobre ellas. Sin embargo, el modelo desarrollado solo podría reconocer las 104 clases presentes en el conjunto de datos, por lo que una aplicación real más amplia requeriría entrenarlo con una mayor variedad de especies.

4. Metodología

4.1. Arquitectura de la red

Para clasificar las 104 clases de flores se utiliza una red neuronal convolucional (CNN), ya que este tipo de red está diseñado para trabajar con imágenes y aprende automáticamente características espaciales como bordes, texturas, colores y formas.

Los experimentos se realizaron con imágenes RGB de 112×112 píxeles, obtenidas al redimensionar los archivos originales de 224×224; por tanto, la entrada de la red es de $112 \times 112 \times 3$. La Tabla 3 resume la arquitectura. Se usan filtros de 3×3 porque analizan regiones pequeñas de la imagen: las primeras capas aprenden características simples, como bordes o cambios de color, y las posteriores las combinan para representar estructuras más complejas de las flores. El número de filtros aumenta de 32 a 256 mientras el *max pooling* reduce el tamaño espacial.

4.2. Preprocesamiento

Cada registro TFRecord de 224×224 se decodifica como imagen RGB de tres canales, se redimensiona a 112×112 y se normaliza al intervalo $[0, 1]$ mediante

$$x_{\text{norm}} = \frac{x}{255}. \tag{3}$$

La reducción de resolución disminuye el costo de entrenamiento y conserva la forma y el color de las flores.

En el conjunto completo se aplica aumento de datos solo durante el entrenamiento: volteo horizontal, rotación de hasta $\pm 18°$, acercamiento o alejamiento de hasta 10 % y variación de contraste de hasta 10 %. Estas transformaciones generan versiones distintas de una misma imagen sin modificar su clase. A validación y prueba no se les aplica ninguna transformación aleatoria, para evaluar siempre las mismas imágenes.

4.3. Función de pérdida y entrenamiento

Cada imagen pertenece a una sola de las 104 clases, por lo que se utiliza la entropía cruzada categórica dispersa. La red produce 104 *logits* z_k, que Softmax convierte en probabilidades:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{104} e^{z_j}}. \tag{4}$$

Para una imagen de clase c, la pérdida es $L = -\log(\hat{y}_c)$: es pequeña si el modelo asigna alta probabilidad a la clase correcta y grande si se equivoca con confianza. Se usa la versión dispersa porque las etiquetas son enteros de 0 a 103 y no hace falta convertirlas a vectores *one-hot*.

Los gradientes se calculan con retropropagación y los parámetros se actualizan con Adam, con una tasa de aprendizaje de 10^{-3} y minilotes de 32 imágenes mezcladas en cada época. Después de cada evaluación se calcula el F1 macro de validación, se guarda el modelo con el mejor F1 (*checkpoint*) y el entrenamiento se detiene tras cinco evaluaciones consecutivas sin mejora (parada temprana).

4.4. Destilación de datos mediante Distribution Matching

Además del entrenamiento convencional, se implementó la destilación de datos mediante *Distribution Matching* (DM) (Zhao & Bilen, 2023). Su objetivo es generar un conjunto reducido de imágenes sintéticas que conserve las características representativas del conjunto original.

El método utiliza una red convolucional como extractor de características. En cada iteración se inicializan nuevos pesos aleatorios y se obtienen las representaciones de imágenes reales y sintéticas de cada clase. La función objetivo minimiza la distancia cuadrática entre los promedios de estas representaciones:

$$L_{DM} = \sum_{c=0}^{103} \left\| \mu_c^{\text{real}} - \mu_c^{\text{sint}} \right\|_2^2, \tag{5}$$

donde μ_c^{real} y μ_c^{sint} son los vectores promedio de características de las imágenes reales y sintéticas de la clase c. Solo se optimizan los píxeles de las imágenes sintéticas; los pesos del extractor permanecen fijos durante cada paso.

Tabla 3
Arquitectura de la CNN (480,936 parámetros entrenables).

Capa	Configuración	Dimensión de salida
Entrada	Imagen RGB	$112 \times 112 \times 3$
Convolución 1	32 filtros de 3×3, *padding same* + ReLU	$112 \times 112 \times 32$
Max Pooling 1	Ventana de 2×2	$56 \times 56 \times 32$
Convolución 2	64 filtros de 3×3, *padding same* + ReLU	$56 \times 56 \times 64$
Max Pooling 2	Ventana de 2×2	$28 \times 28 \times 64$
Convolución 3	128 filtros de 3×3, *padding same* + ReLU	$28 \times 28 \times 128$
Max Pooling 3	Ventana de 2×2	$14 \times 14 \times 128$
Convolución 4	256 filtros de 3×3, *padding same* + ReLU	$14 \times 14 \times 256$
Max Pooling 4	Ventana de 2×2	$7 \times 7 \times 256$
Global Average Pooling	Promedio de cada mapa de características	256
Capa densa	256 neuronas + ReLU	256
Dropout	Tasa de 0.5	256
Salida	104 neuronas (*logits*), una por clase	104

Se utilizó Adam con una tasa de aprendizaje de 0.01 sobre los píxeles durante 500 iteraciones completas, con transformaciones diferenciables de aumento de datos (DSA) (Zhao & Bilen, 2021) sobre las imágenes reales y sintéticas. Se generaron conjuntos de una y diez imágenes por clase (IPC = 1 e IPC = 10), que después se usaron para entrenar la CNN; durante ese entrenamiento también se aplica DSA.

5. Entrenamiento y resultados

En esta etapa se entrenó la CNN con el conjunto completo y se evaluó el efecto de reducir el número de imágenes disponibles por clase, comparando subconjuntos aleatorios de fotografías reales con conjuntos sintéticos obtenidos con DM. Después, con el conjunto completo, se probaron variaciones de hiperparámetros. Todas las tablas y figuras de esta sección se generan con el notebook `reports/entregable_3/avance3.ipynb` del repositorio público del proyecto (Eduardo-Luna et al., 2026) (sección 5.6).

5.1. Diseño de los experimentos

Se consideraron tres modalidades de entrenamiento. La primera, *conjunto completo*, emplea las 12,753 imágenes reales de entrenamiento y funciona como referencia. La segunda, *aleatorio*, utiliza un subconjunto balanceado de fotografías reales tomadas al azar, con la misma cantidad de imágenes por clase. La tercera, *destilado*, sustituye ese subconjunto por imágenes sintéticas optimizadas con DM. En las dos últimas se evaluaron dos tamaños: una imagen por clase (IPC = 1, 104 imágenes) y diez imágenes por clase (IPC = 10, 1,040 imágenes), que representan reducciones del 99.18 % y 91.85 % frente al conjunto original.

Las cuatro configuraciones reducidas se entrenaron con las semillas 0, 1 y 2, hasta 300 épocas, evaluando validación cada diez épocas. El conjunto completo se entrenó una vez (semilla 0), hasta 50 épocas, evaluando validación en cada época. En todos los casos se usó el mismo conjunto de validación de 3,712 imágenes reales; los datos de prueba no se usaron para seleccionar modelos.

5.2. Resultados de clasificación

La Tabla 4 resume los resultados. Para los conjuntos reducidos se presenta el promedio de las tres semillas y la desviación estándar muestral del F1 macro. La exactitud se incluye como métrica complementaria, pero el F1 macro es la medida principal por el desbalance entre clases.

Con una sola imagen por clase, el conjunto destilado alcanzó un F1 macro promedio de 0.0672, frente a 0.0514 con imágenes reales elegidas al azar, una diferencia de 0.0158 a favor de DM. Esto sugiere que, con una reducción extrema, la optimización de las imágenes sintéticas conserva información útil para la clasificación. Con diez imágenes por clase ocurrió lo contrario: la selección aleatoria alcanzó 0.2575 y el conjunto destilado 0.2044, una diferencia de 0.0531 a favor de las fotografías reales (Figura 3). Por tanto, la destilación no mostró una ventaja consistente en ambos tamaños.

La CNN entrenada con el conjunto completo obtuvo el mejor resultado: F1 macro de 0.5538 y exactitud de 61.61 %. Como el modelo completo se entrenó una sola vez y las demás configuraciones cuentan con tres repeticiones, estas comparaciones son resultados preliminares, no evidencia estadística definitiva.

5.3. Costo computacional de la destilación

La Tabla 5 separa el tiempo de entrenamiento del tiempo de destilación. Los tiempos de entrenamiento excluyen las evaluaciones periódicas de validación. La destilación se ejecutó una sola vez por IPC y sus imágenes se reutilizaron en las tres semillas, por lo que no debe contarse una destilación por semilla. La columna "Total" suma una destilación al tiempo promedio de entrenamiento; estima el costo de preparar el conjunto sintético y entrenar un modelo con él, pero no incluye la descarga ni la preparación de los datos.

Aunque entrenar con conjuntos reducidos toma menos tiempo que usar las 12,753 fotografías, la destilación incrementa considerablemente el costo inicial: con IPC = 10, preparar y entrenar el conjunto destilado (1,782.5 s) toma más que entrenar con el conjunto completo (581.5 s). Por

Tabla 4
Resultados de validación y tiempo de entrenamiento de la CNN a 112×112. Para los conjuntos reducidos se reportan promedios de tres semillas.

Modalidad	IPC	Corridas	F1 macro	Exactitud (%)	Entrenamiento (s)
Aleatorio	1	3	0.0514 ± 0.0029	5.75	35.4
Destilado (DM)	1	3	0.0672 ± 0.0052	9.85	36.3
Aleatorio	10	3	0.2575 ± 0.0082	25.86	329.4
Destilado (DM)	10	3	0.2044 ± 0.0235	20.55	272.5
Conjunto completo	–	1	0.5538	61.61	581.5

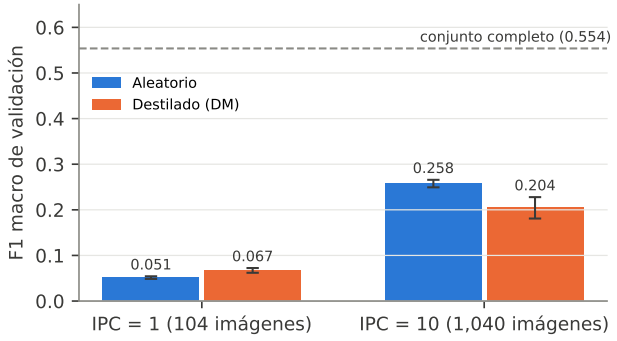


Figura 3: F1 macro de validación de los subconjuntos aleatorios y destilados. Las barras de error indican una desviación estándar entre tres semillas y la línea discontinua, la corrida con el conjunto completo.

Tabla 5
Tiempo promedio de entrenamiento y de destilación previa, en segundos.

Modalidad	IPC	Entren.	Destil.	Total
Aleatorio	1	35.4	–	35.4
Destilado (DM)	1	36.3	985.4	1,021.7
Aleatorio	10	329.4	–	329.4
Destilado (DM)	10	272.5	1,510.0	1,782.5
Conjunto completo	–	581.5	–	581.5

ello, los resultados no permiten concluir que DM sea más rápido para entrenar un solo modelo; su costo solo podría amortizarse si las mismas imágenes sintéticas se reutilizaran en muchos entrenamientos.

5.4. Curvas de aprendizaje

Cada corrida guarda un archivo `history.csv` con la pérdida y la exactitud de entrenamiento y la pérdida, exactitud y F1 macro de validación por época. Las métricas de entrenamiento se registran durante la época, con aumento de datos y *dropout* activos, mientras que las de validación se calculan al final de la época sin transformaciones. Por eso, al inicio, la exactitud de validación puede ser mayor que la de entrenamiento.

Conjunto completo. La Figura 4 muestra que las pérdidas de entrenamiento y validación bajan juntas hasta la época 21 (de 3.94 y 3.59 a 1.69 y 1.74). A partir de la época 25 se separan: la de entrenamiento sigue bajando hasta 1.07, mientras que la de validación se estanca entre 1.51 y 1.69. La exactitud muestra lo mismo: al final, la de entrenamiento llega a 68.1 % y la de validación se queda en 62.6 %. Es el inicio del sobreajuste, aunque moderado, porque el F1 de validación todavía mejora lentamente hasta la época 38 (0.5538), que coincide con la menor pérdida de validación (1.510). En las cinco épocas siguientes el F1 no mejoró, y la parada temprana detuvo el entrenamiento en la época 43. El *checkpoint* guardado corresponde a la época 38, así que el modelo reportado no incluye las épocas en las que empezaba a sobreajustar.

Conjuntos reducidos. En la Figura 5, el sobreajuste es mucho más marcado. Con IPC = 10, la exactitud de entrenamiento llega a 70–85 % y la pérdida de entrenamiento baja a 0.5–0.9, pero la exactitud de validación se estanca alrededor de 25 % (aleatorio) y 19 % (destilado). La pérdida de validación alcanza su mínimo entre las épocas 30 y 50 (alrededor de 3.6–3.9) y luego crece hasta 7.4–9.3, es decir, la red memoriza las 1,040 imágenes. Con IPC = 1 ocurre lo mismo con mayor intensidad: la validación queda entre 5 y 10 % de exactitud y su pérdida sube hasta 12–18. El conjunto destilado con IPC = 1 se ajusta más lentamente (exactitud de entrenamiento final de 57–63 %, contra 62–83 % del aleatorio), lo que concuerda con que sus imágenes sintéticas resumen la clase en lugar de ser ejemplos fáciles de memorizar.

Aunque la pérdida de validación crece, el mejor F1 de los conjuntos reducidos se alcanza tarde (entre las épocas 170 y 280). Esto pasa porque la pérdida castiga las predicciones equivocadas hechas con mucha confianza, mientras que el F1 solo depende de la clase predicha: el modelo sigue acertando algunas imágenes más, pero se vuelve muy confiado en sus errores. Por eso conviene seleccionar el modelo por F1 y no por la pérdida.

5.5. Experimentos de hiperparámetros con el conjunto completo

Para estudiar los hiperparámetros, se tomó como referencia una política de entrenamiento con AdamW (*weight decay* de 0.01), tasa de aprendizaje inicial de 10^{-3} que se

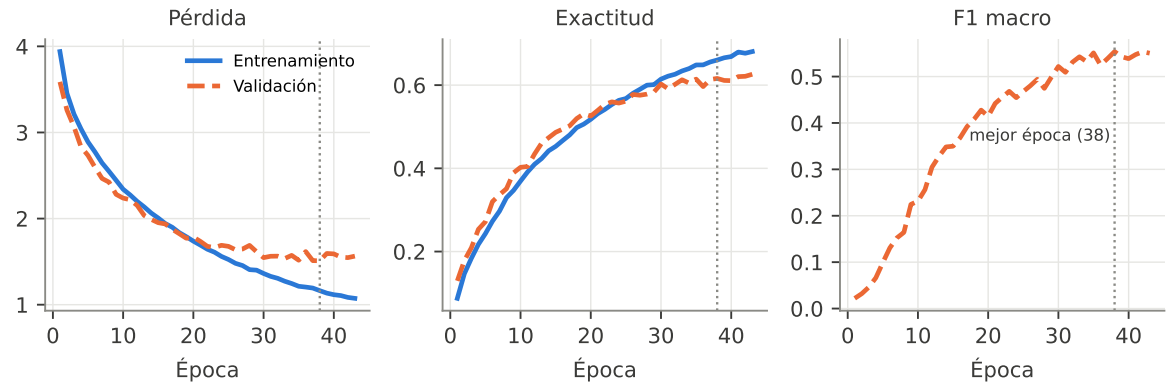


Figura 4: Curvas de aprendizaje con el conjunto completo: pérdida y exactitud de entrenamiento frente a validación, y F1 macro de validación. La línea punteada marca la mejor época (38), cuyo modelo se guardó.

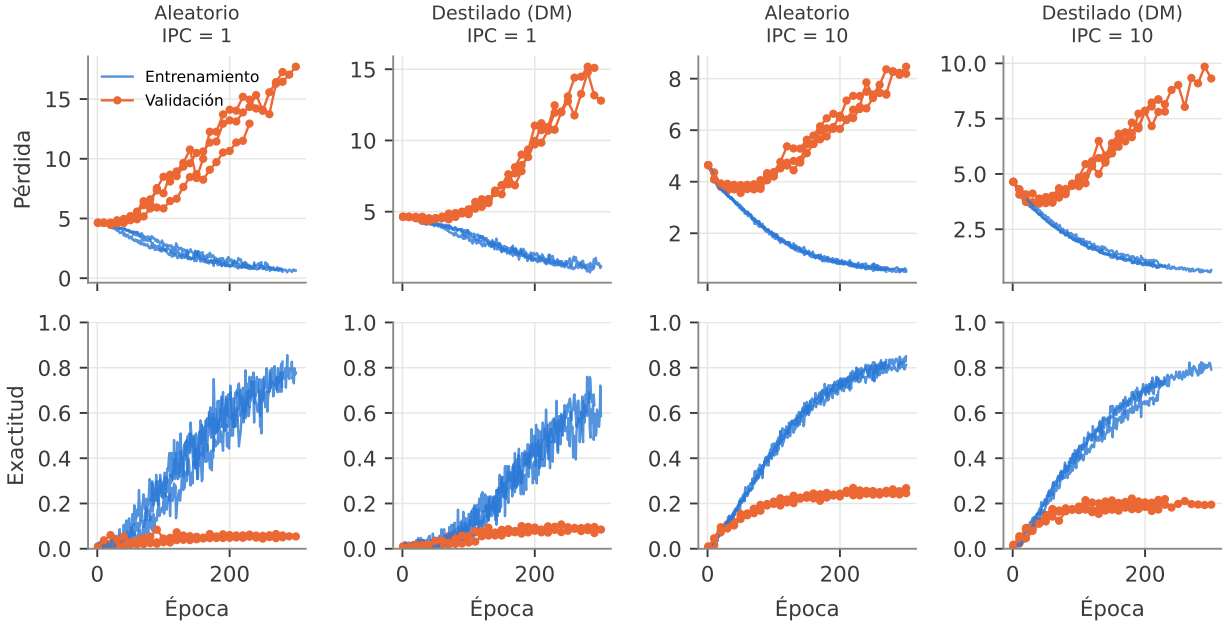


Figura 5: Curvas de aprendizaje de los conjuntos reducidos (tres semillas por configuración). Validación se evaluó cada diez épocas; los marcadores indican las épocas evaluadas.

reduce a la mitad cuando la pérdida de validación se estanca, *dropout* de 0.5, aumento de datos, lotes de 64 y paciencia de 12 épocas. A partir de ella se cambió **un solo factor** en cada experimento: la tasa de aprendizaje (3×10^{-4}), el *dropout* (0.3) y la regularización en conjunto (sin *dropout*, sin aumento de datos y sin *weight decay*). En estas corridas, las métricas de entrenamiento se miden sobre 3,712 imágenes fijas de entrenamiento sin aumentos ni *dropout*, igual que validación, por lo que la brecha entre ambas sí mide el sobreajuste. Los resultados están en la Tabla 6 y la Figura 6.

Tasa de aprendizaje. Reducir la tasa de 10^{-3} a 3×10^{-4} bajó el F1 de 0.5784 a 0.4605. La brecha es la más pequeña (0.094), pero el F1 de entrenamiento también es bajo (0.554): el modelo no está sobreajustando, está **subajustando**. Con pasos más pequeños, la red aprende más lento y la reducción automática de la tasa la lleva al mínimo (10^{-5}) antes de converger, así que después de 110 épocas sigue lejos de la referencia. Con este presupuesto de épocas, 10^{-3} es la mejor opción.

Dropout. Bajar el *dropout* de 0.5 a 0.3 redujo el F1 a 0.5596 y aumentó ligeramente la brecha (de 0.235 a 0.256). La menor pérdida de validación también empeoró (de 1.464 a 1.596). Con menos neuronas apagadas, la capa densa memoriza un poco más, así que conviene mantener 0.5.

Regularización. Sin *dropout*, sin aumento de datos y sin *weight decay*, la red converge más rápido (mejor época 34 de 46), pero alcanza la brecha más grande (0.344): el F1 de entrenamiento llega a 0.877, mientras que el de validación se queda en 0.5332. Es el caso más claro de **sobreajuste**, lo

Tabla 6
Efecto de los hiperparámetros con el conjunto completo (una semilla). La brecha es el F1 de entrenamiento menos el de validación en la mejor época.

Configuración	F1 val.	Exactitud val. (%)	F1 entren.	Brecha	Mejor época / épocas
Referencia (LR 10^{-3}, dropout 0.5)	0.5784	64.06	0.814	0.235	91 / 103
Tasa de aprendizaje 3×10^{-4}	0.4605	54.18	0.554	0.094	98 / 110
Dropout 0.3	0.5596	61.96	0.816	0.256	61 / 73
Sin dropout, aumento ni weight decay	0.5332	59.64	0.877	0.344	34 / 46

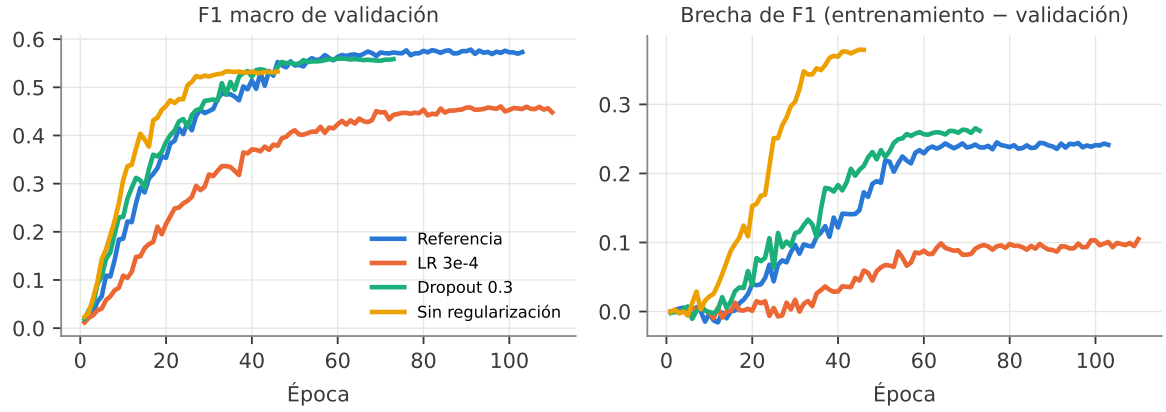


Figura 6: F1 macro de validación y brecha de F1 entre entrenamiento y validación por época para cada configuración de hiperparámetros.

que confirma que la regularización es la responsable de la mejora de la referencia.

La referencia (F1 de 0.5784, exactitud de 64.06 %) supera a la corrida original del Avance 3 (0.5538). Sin embargo, como esa política cambia varios factores a la vez (optimizador, reducción de la tasa, tamaño de lote y paciencia), la mejora no puede atribuirse a uno solo. No se evaluó el tamaño de imagen; queda como siguiente experimento, por ejemplo 160×160.

5.6. Notebook y reproducibilidad

El notebook `reports/entregable_3/avance3.ipynb` del repositorio https://github.com/Lunaaaalj/NeuralNetworksTeamEnvironment reproduce esta sección de principio a fin: descarga y revisión de los datos, construcción de la CNN, entrenamiento de todas las configuraciones (activando `RUN_TRAINING`), re-evaluación de los modelos guardados, generación de las Tablas 4, 5 y 6 y de todas las figuras de curvas. La configuración exacta del Avance 3 está en `config_avance3.yaml` y la de cada experimento de hiperparámetros, junto con su historial, en `hiperparametros/`. Al re-evaluar los 13 modelos guardados sobre las 3,712 imágenes de validación, el F1 macro y la exactitud coincidieron exactamente con los registrados durante el entrenamiento.

5.7. Limitaciones y siguientes pasos

El tamaño y la composición del conjunto de entrenamiento influyen mucho en el desempeño de la CNN. DM mejoró a la selección aleatoria con una imagen por clase, pero no con diez, y el costo de crear las imágenes sintéticas superó el de entrenar con ellas. Con el conjunto completo, la regularización (*dropout* de 0.5, aumento de datos y *weight decay*) y una tasa de aprendizaje de 10^{-3} dieron el mejor resultado.

Las conclusiones están limitadas a una arquitectura, una resolución de 112×112, una sola semilla en los experimentos con el conjunto completo y al conjunto de validación, ya que las etiquetas de prueba no están disponibles. Los siguientes pasos son probar resoluciones mayores, repetir los experimentos de hiperparámetros con varias semillas y estudiar qué clases son más difíciles de reconocer.

Referencias

Eduardo-Luna, A., De la Peña Contreras, A. E., Flores Rodríguez, J. C., de la Fuente Chávez, R. I., & Huerta Robinson, A. (2026). *NeuralNetworksTeamEnvironment: clasificación de 104 tipos de flores*. Consultado el 25 de octubre de 2026, desde https://github.com/Lunaaaalj/NeuralNetworksTeamEnvironment

IBM. (s.f.). *What is image classification?* Consultado el 28 de septiembre de 2026, desde https://www.ibm.com/think/topics/image-classification

iNaturalist. (2026). *iNaturalist: A community for naturalists*. Consultado el 28 de septiembre de 2026, desde https://www.inaturalist.org

Kaggle. (2020). *Petals to the Metal: Flower Classification on TPU*. Consultado el 28 de septiembre de 2026, desde https://www.kaggle.com/competitions/tpu-getting-started/overview

Nilsback, M.-E., & Zisserman, A. (2008). Automated flower classification over a large number of classes. *2008 Sixth Indian Conference*

on Computer Vision, Graphics & Image Processing, 722-729. https://doi.org/10.1109/ICVGIP.2008.47

Zhao, B., & Bilen, H. (2021). Dataset condensation with differentiable siamese augmentation. *Proceedings of the 38th International Conference on Machine Learning*, *139*, 12674-12685. Consultado el 25 de octubre de 2026, desde https://proceedings.mlr.press/v139/zhao21a.html

Zhao, B., & Bilen, H. (2023). Dataset condensation with distribution matching. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 6514-6523. Consultado el 25 de octubre de 2026, desde https://openaccess.thecvf.com/content/WACV2023/html/Zhao_Dataset_Condensation_With_Distribution_Matching_WACV_2023_paper.html